

SAS Retrieval Agent Manager (RAM) v1 – Complete REST API Specification

Portal Source: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager>

Total Endpoints: 24

Request Samples: JavaScript (Fetch), Python (Requests & HTTPX), Shell (cURL)

Response Samples: JSON Payloads for HTTP 200, 201, 400, 401, 422

Table of Contents

- [Root](#) (1 endpoints)
 - [Query](#) (2 endpoints)
 - [Query Sessions](#) (1 endpoints)
 - [Collections](#) (1 endpoints)
 - [Llms](#) (1 endpoints)
 - [Llm Calls](#) (1 endpoints)
 - [Tool Calls](#) (1 endpoints)
 - [Retrieval Calls](#) (1 endpoints)
 - [Agents](#) (1 endpoints)
 - [Sources](#) (11 endpoints)
 - [Tags](#) (3 endpoints)
-

Root

GET /

Summary: Get a list of the public endpoints

Operation ID: `root`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/root>

Returns a list of the public endpoints.

Request Samples

JavaScript - Fetch

```

const url = 'https://example.com/SASRetrievalAgentManager/api/v1/';
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",

```

```
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	Successful Response	application/json
401	The user was not authenticated.	application/json

Response Samples

200 application/json (Successful Response)

```
{
  "links": [
    {
      "href": "/",
      "method": "GET",
      "rel": "self",
      "responseType": "application/json",
      "type": "application/json",
      "uri": "/"
    }
  ],
  "version": "1"
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

Query

GET /query

Summary: Get a list of the available queries

Operation ID: `getQueries`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getQueries>

Returns a list of all the queries for a given session ID.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include <code>id</code> , <code>querySessionId</code> , and <code>parentQueryId</code> . Additional attributes might be added in future releases.
<code>limit</code>	query	<code>integer</code>	No	The maximum number of records to be returned. <code>>= 1</code> or <code><= 100</code>
<code>start</code>	query	<code>integer</code>	No	The zero-based offset of the first record to be returned. <code>>= 0</code>

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/query';
const options = {
```

```

method: 'GET',
headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/query"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/query"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
  }
  async with httpx.AsyncClient() as client:

```

```
response = await client.get(url, headers=headers)
print(response.json())
```

```
asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/query" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "content": "What is the weather today?",
      "errorCode": 0,
      "errorText": "",
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "origin": "user",
```

```

    "parentQueryId": "",
    "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1",
    "response": {
      "answer": "The weather today is sunny with a high of 75\u00b0F.",
      "context": [],
      "toolCalls": [],
      "usageMetadata": {
        "llmCompletionCost": 0.00024,
        "llmCompletionTokens": 12,
        "llmPromptCost": 0.00016,
        "llmPromptTokens": 8,
        "llmTotalCost": 0.0004,
        "llmTotalTokens": 20
      }
    },
    "target": "collection",
    "targetId": {
      "configuration_ids": [
        "86cc0aae-04ed-4e0f-97d2-53e8b7dc9881"
      ]
    }
  }
],
"limit": 50,
"name": "queries",
"start": 0
}

```

400 application/json (The request was invalid.)

```

{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}

```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

POST /query

Summary: Create a query

Operation ID: `createQuery`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/createQuery>

Creates a query.

Parameters

Name	Location	Type	Required	Description
<code>synchronous</code>	query	<code>boolean</code>	Yes	This is an indicator for whether the query response is returned. If 'true', this call returns the query response. If 'false', the query is executed asynchronously and the response must be fetched separately. This parameter is required.

Name	Location	Type	Required	Description
persistent	query	boolean	No	This is an indicator for whether to persist the query in the SAS Retrieval Agent Manager database. If 'true', the query persists. If 'false', the query is not retained in the database.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/query';
const options = {
  method: 'POST',
  headers: {
    Authorization: 'Bearer YOUR_ACCESS_TOKEN',
    'Content-Type': 'application/json',
    Accept: 'application/json'
  },
  body: JSON.stringify({
    "agentId": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
    "content": "What is the weather like today?",
    "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
  })
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/query"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
```

```

        "Content-Type": "application/json",
    }
    payload = {
        "agentId": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
        "content": "What is the weather like today?",
        "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
    }

    try:
        response = requests.post(url, headers=headers, json=payload)
        response.raise_for_status()
        data = response.json()
        print(data)
    except requests.exceptions.RequestException as error:
        print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/query"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
        "Content-Type": "application/json",
    }
    payload = {
        "agentId": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
        "content": "What is the weather like today?",
        "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
    }

    async with httpx.AsyncClient() as client:
        response = await client.post(url, headers=headers, json=payload)
        print(response.json())

asyncio.run(main())

```

Shell - cURL

```
curl -X POST "https://example.com/SASRetrievalAgentManager/api/v1/query"
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{
  "agentId": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "content": "What is the weather like today?",
  "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
}'
```

Request Body – An example request by the user to query an agent

```
{
  "agentId": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "content": "What is the weather like today?",
  "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
}
```

Request Body – An example request by the user to query a collection

```
{
  "collectionIds": [
    "079b46b9-fbc4-44ce-8cb1-163330e4d812",
    "86cc0aae-04ed-4e0f-97d2-53e8b7dc9881"
  ],
  "content": "What is the weather like today?",
  "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
}
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
201	A query was created.	application/json

Status Code	Description	Content-Types
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "content": "What is a business result?",
  "errorCode": 0,
  "id": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "origin": "agent",
  "parentQueryId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "querySessionId": "21647604-55c0-4283-b5df-7e1c273d86aa",
  "response": {
    "answer": "A business result is a measurable outcome that indicates t",
    "context": [],
    "toolCalls": [],
    "usageMetadata": {
      "llmCompletionCost": 0.00056,
      "llmCompletionTokens": 28,
      "llmPromptCost": 0.00016,
      "llmPromptTokens": 8,
      "llmTotalCost": 0.00072,
      "llmTotalTokens": 36
    }
  },
  "target": "collection",
  "targetId": {
    "configurationIds": [
      "86cc0aae-04ed-4e0f-97d2-53e8b7dc9881",
      "079b46b9-fbc4-44ce-8cb1-163330e4d812"
    ]
  }
},
```

```
"userEmail": "appadmin@example.com"
}
```

201 application/json (A query was created.)

```
{
  "content": "What is a business result?",
  "errorCode": 0,
  "id": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "origin": "agent",
  "parentQueryId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "querySessionId": "21647604-55c0-4283-b5df-7e1c273d86aa",
  "response": {
    "answer": "A business result is a measurable outcome that indicates t",
    "context": [],
    "toolCalls": [],
    "usageMetadata": {
      "llmCompletionCost": 0.00056,
      "llmCompletionTokens": 28,
      "llmPromptCost": 0.00016,
      "llmPromptTokens": 8,
      "llmTotalCost": 0.00072,
      "llmTotalTokens": 36
    }
  },
  "target": "collection",
  "targetId": {
    "configurationIds": [
      "86cc0aae-04ed-4e0f-97d2-53e8b7dc9881",
      "079b46b9-fbc4-44ce-8cb1-163330e4d812"
    ]
  },
  "userEmail": "appadmin@example.com"
}
```

201 application/json (A query was created.)

```
{
  "queryId": "6fd000e3-5945-40cf-83ef-0c7fc19fcde1",
  "querySessionId": "4fd000e3-5945-40cf-83ef-0c7fc19fcde1"
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Query Sessions

GET /querySessions

Summary: Get a list of the available query sessions

Operation ID: `getQuerySessions`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getQuerySessions>

Returns a list of the available query sessions.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include id and title. Additional attributes might be added in future releases.
<code>limit</code>	query	<code>integer</code>	No	The maximum number of records to be returned. <code>>= 1</code> or <code><= 100</code>
<code>start</code>	query	<code>integer</code>	No	The zero-based offset of the first record to be returned. <code>>= 0</code>
<code>sortBy</code>	query	<code>['string', 'null']</code>	No	The criteria for sorting. Supported attributes include id, title, insertTimestamp, and updateTimestamp. Additional attributes might be added in future releases.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/querySessions';
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'application/json'};
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/querySessions"
headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
}

try:
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/querySessi"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/querySes" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "insertTimestamp": "2025-12-12T19:48:53.448286+00:00",
      "title": "What is the weather today?",
      "updateTimestamp": "2025-12-12T19:48:53.448286+00:00"
    }
  ],
  "limit": 50,
  "name": "querySessions",
  "start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
}
```

```
"errorCode": 400,  
"httpStatusCode": 400,  
"message": "Invalid query parameter.",  
"version": 2  
}
```

401 application/json (The user was not authenticated.)

```
{  
  "details": [  
    "correlator: 123e4567-e89b-12d3-a456-426614174000"  
  ],  
  "errorCode": 401,  
  "httpStatusCode": 401,  
  "message": "The credentials could not be validated.",  
  "version": 2  
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{  
  "details": [  
    "correlator: 123e4567-e89b-12d3-a456-426614174000"  
  ],  
  "errorCode": 422,  
  "httpStatusCode": 422,  
  "message": "Missing or invalid fields in request body or parameters.",  
  "version": 2  
}
```

Collections

GET /collections

Summary: Get a list of the available collections

Operation ID: `getCollections`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getCollections>

Returns a list of the available collections.

Parameters

Name	Location	Type	Required	Description
filter	query	['string', 'null']	No	The criteria for filtering. Supported attributes include id and name. Additional attributes might be added in future releases.
limit	query	integer	No	The maximum number of records to be returned. ≥ 1 or ≤ 100
start	query	integer	No	The zero-based offset of the first record to be returned. ≥ 0

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/collections';
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'application/json'};
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/collections"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}
```

```

try:
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/collection"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())

```

Shell - cURL

```

curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/collection" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"

```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json

Status Code	Description	Content-Types
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "description": "A collection containing weather data.",
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "name": "Weather Collection"
    }
  ],
  "limit": 50,
  "name": "collections",
  "start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Llms

GET /llms

Summary: Get a list of the available LLMs

Operation ID: `getLlms`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getLlms>

Returns a list of all the available LLMs.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include id and name. Additional attributes might be added in future releases.

Name	Location	Type	Required	Description
limit	query	integer	No	The maximum number of records to be returned. ≥ 1 or ≤ 100
start	query	integer	No	The zero-based offset of the first record to be returned. ≥ 0

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/llms';
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'application/json'}
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/llms"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
```

```
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/llms"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/llms" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json

Status Code	Description	Content-Types
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "description": "LLM From API",
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "name": "gpt-4o",
      "type": "azure_openai"
    }
  ],
  "limit": 50,
  "name": "llms",
  "start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
```

```
    "correlator": 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator": 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Llm Calls

GET /llmCalls

Summary: Get a list of the available LLM calls

Operation ID: `getLlmCalls`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getLlmCalls>

Returns a list of the available LLM calls.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include <code>id</code> , <code>llmId</code> , and <code>parentQueryId</code> . Additional attributes might be added in future releases.
<code>limit</code>	query	<code>integer</code>	No	The maximum number of records to be returned. <code>>= 1</code> or <code><= 100</code>

Name	Location	Type	Required	Description
start	query	integer	No	The zero-based offset of the first record to be returned. ≥ 0

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/llmCalls'
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/llmCalls"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/llmCalls"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/llmCalls" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "completionCost": 0.0005,
      "completionTokens": 25,
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "input": {
        "content": "What is the weather like today?",
        "modelName": "gpt-4o",
        "modelProvider": "azure",
        "temperature": 0.7
      },
      "llmId": "de772bda-8756-41a8-9c68-defe2932bdad",
      "output": {
        "response": "The weather today is sunny with a high of 75\u00b0F.",
        "toolCalls": []
      },
      "parentQueryId": "19c0e653-28f5-49da-aa34-7ade481b362e",
      "promptCost": 0.0003,
      "promptTokens": 15
    }
  ],
  "limit": 50,
  "name": "collections",
  "start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Tool Calls

GET /toolCalls

Summary: Get a list of all tool calls

Operation ID: `getToolCalls`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getToolCalls>

Returns a list of the tool calls.

Parameters

Name	Location	Type	Required	Description
filter	query	['string', 'null']	No	The criteria for filtering. Supported attributes include id, toolServerId, toolName and parentQueryId. Additional attributes might be added in future releases.
limit	query	integer	No	The maximum number of records to be returned. ≥ 1 or ≤ 100
start	query	integer	No	The zero-based offset of the first record to be returned. ≥ 0

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/toolCall
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/toolCalls"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
```

```
response = requests.get(url, headers=headers)
response.raise_for_status()
data = response.json()
print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/toolCalls"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/toolCall" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json

Status Code	Description	Content-Types
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "cost": 0.0001,
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "input": {
        "question": "Is the openai/gym repository related to AI agents?"
      },
      "output": {
        "content": [
          {
            "text": "Yes, the openai/gym repository is directly related t",
            "type": "text"
          }
        ],
        "isError": false,
        "structuredContent": {
          "result": "Yes, the openai/gym repository is directly related t"
        }
      },
      "parentQueryId": "19c0e653-28f5-49da-aa34-7ade481b362e",
      "toolName": "ask_question",
      "toolServerId": "de772bda-8756-41a8-9c68-defe2932bdad"
    }
  ],
  "limit": 50,
  "name": "toolCalls",
}
```

```
"start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Retrieval Calls

GET /retrievalCalls

Summary: Get a list of the available retrieval calls

Operation ID: `getRetrievalCalls`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getRetrievalCalls>

Returns a list of the retrieval calls made.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include <code>id</code> and <code>parentQueryId</code> . Additional attributes might be added in future releases.
<code>limit</code>	query	<code>integer</code>	No	The maximum number of records to be returned. <code>>= 1</code> or <code><= 100</code>
<code>start</code>	query	<code>integer</code>	No	The zero-based offset of the first record to be returned. <code>>= 0</code>

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/retrievalCalls';
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'application/json'}
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/retrievalCalls"
headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
}

try:
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/retrievalC"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/retrieva" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "completionCost": 0.0005,
      "completionTokens": 25,
      "configurationIds": [
        "de772bda-8756-41a8-9c68-defe2932bdad"
      ],
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "input": {
        "content": "What is the weather like today?"
      },
      "output": {
        "documents": [
          {
            "id": "weather-document-123",
            "metadata": {},
            "pageContent": "The weather today is sunny with a high of 75\\",
            "type": "Document"
          }
        ]
      }
    }
  ]
}
```

```
    },
    "parentQueryId": "19c0e653-28f5-49da-aa34-7ade481b362e"
  }
],
"limit": 50,
"name": "retrievalCalls",
"start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
}
```

```
"message": "Missing or invalid fields in request body or parameters.",
"version": 2
}
```

Agents

GET /agents

Summary: Get a list of all agents

Operation ID: `getAgents`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getAgents>

Returns a list of the agents.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include id and name. Additional attributes might be added in future releases.
<code>limit</code>	query	<code>integer</code>	No	The maximum number of records to be returned. <code>>= 1</code> or <code><= 100</code>
<code>start</code>	query	<code>integer</code>	No	The zero-based offset of the first record to be returned. <code>>= 0</code>

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/agents';
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
```

```
const data = await response.json();
console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/agents"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/agents"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
  }
  async with httpx.AsyncClient() as client:
    response = await client.get(url, headers=headers)
    print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/agents"
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 1,
  "items": [
    {
      "description": "An agent providing weather data.",
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "name": "Weather Agent"
    }
  ],
  "limit": 50,
  "name": "agents",
  "start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Sources

GET /sources

Summary: Get a list of the available sources

Operation ID: `getSources`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getSources>

Returns a list of sources.

Parameters

Name	Location	Type	Required	Description
<code>limit</code>	query	<code>['integer', 'null']</code>	No	The maximum number of sources to be returned.
<code>start</code>	query	<code>['integer', 'null']</code>	No	The zero-based offset of the first source to be returned.
<code>sortBy</code>	query	<code>['string', 'null']</code>	No	The criteria for sorting. Supported attributes include name, insertTimestamp, and updateTimestamp. Additional attributes might be added in future releases.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources'
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources"
headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
}

try:
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())

```

Shell - cURL

```

curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/sources" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"

```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 3,
  "items": [
    {
      "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
      "inputType": "file",
      "insertTimestamp": "2025-12-12T19:48:53.448286+00:00",
      "localUri": "/ramuser/data/infoFiles",
      "metadata": {
        "description": "Collected info files for RAM source."
      },
      "name": "Info Files",
      "sourceType": "local",
      "updateTimestamp": "2025-12-13T09:15:22.123456+00:00"
    },
    {
      "id": "86cc0aae-04ed-4e0f-97d2-53e8b7dc9881",
      "inputType": "file",
      "insertTimestamp": "2025-10-05T08:12:00.000000+00:00",
      "localUri": "/ramuser/data/weather",
      "metadata": {
        "description": "Daily weather CSV files."
      },
      "name": "Weather Data",
```

```
    "sourceType": "local",
    "updateTimestamp": "2025-11-15T17:45:00.000000+00:00"
  },
  {
    "id": "de772bda-8756-41a8-9c68-defe2932bdad",
    "inputType": "file",
    "insertTimestamp": "2025-09-01T12:00:00.000000+00:00",
    "localUri": "/ramuser/data/gitDocs",
    "metadata": {
      "description": "Documentation synced from remote git."
    },
    "name": "Git Docs",
    "sourceType": "git",
    "updateTimestamp": "2025-12-20T12:30:00.000000+00:00"
  }
],
"limit": 10,
"name": "sources",
"start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
}
```

```
"version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

POST /sources

Summary: Create a source

Operation ID: `createSource`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/createSource>

Creates a source.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources'
const options = {
  method: 'POST',
  headers: {
    Authorization: 'Bearer YOUR_ACCESS_TOKEN',
    'Content-Type': 'application/json',
    Accept: 'application/json'
  },
  body: JSON.stringify({
    "description": "Files with much info",
    "inputType": "file",
    "metadata": {},
    "name": "Info Files",
    "sourceType": "local"
  })
}
```

```

}))
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
  "Content-Type": "application/json",
}
payload = {
  "description": "Files with much info",
  "inputType": "file",
  "metadata": {},
  "name": "Info Files",
  "sourceType": "local"
}

try:
  response = requests.post(url, headers=headers, json=payload)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

```

```

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
        "Content-Type": "application/json",
    }
    payload = {
        "description": "Files with much info",
        "inputType": "file",
        "metadata": {},
        "name": "Info Files",
        "sourceType": "local"
    }

    async with httpx.AsyncClient() as client:
        response = await client.post(url, headers=headers, json=payload)
        print(response.json())

asyncio.run(main())

```

Shell - cURL

```

curl -X POST "https://example.com/SASRetrievalAgentManager/api/v1/sources" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{
  "description": "Files with much info",
  "inputType": "file",
  "metadata": {},
  "name": "Info Files",
  "sourceType": "local"
}'

```

Request Body – An example of a local source object

```

{
  "description": "Files with much info",
  "inputType": "file",
  "metadata": {},
  "name": "Info Files",

```

```
"sourceType": "local"
}
```

Request Body – An example of a git source object

```
{
  "description": "A git repository source",
  "inputType": "file",
  "metadata": {
    "schedule": "0 0 * * 5"
  },
  "name": "Git Repository",
  "remoteUri": "https://github.com/example/repo.git",
  "sourceType": "git"
}
```

Response Status Codes

Status Code	Description	Content-Types
201	The source was created.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

201 application/json (The source was created.)

```
{
  "description": "Files with much info",
  "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
  "inputType": "file",
  "name": "Example Source",
  "remoteUri": "https://github.com/example/repo.git",
  "sourceType": "git"
}
```

```
"name": "Info Files",
"sourceType": "local"
}
```

201 application/json (The source was created.)

```
{
  "description": "A git repository source",
  "id": "86cc0aae-04ed-4e0f-97d2-53e8b7dc9881",
  "inputType": "repository",
  "name": "Git Repository",
  "sourceType": "git"
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

DELETE /sources/{sourceId}

Summary: Delete a source

Operation ID: deleteSource

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/deleteSource>

Deletes a source object by its unique identifier.

Parameters

Name	Location	Type	Required	Description
sourceId	path	string	Yes	The id for the source to delete.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/';
const options = {
  method: 'DELETE',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
```

```
    console.error(error);  
}
```

Python - Requests

```
import requests  
  
url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"  
headers = {  
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",  
    "Accept": "application/json",  
}  
  
try:  
    response = requests.delete(url, headers=headers)  
    response.raise_for_status()  
    data = response.json()  
    print(data)  
except requests.exceptions.RequestException as error:  
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx  
import asyncio  
  
async def main():  
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"  
    headers = {  
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",  
        "Accept": "application/json",  
    }  
    async with httpx.AsyncClient() as client:  
        response = await client.delete(url, headers=headers)  
        print(response.json())  
  
asyncio.run(main())
```

Shell - cURL

```
curl -X DELETE "https://example.com/SASRetrievalAgentManager/api/v1/source/  
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \  
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
204	The source was deleted.	None
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

400 application/json (The request was invalid.)

```
{  
  "details": [  
    "correlator: 123e4567-e89b-12d3-a456-426614174000"  
  ],  
  "errorCode": 400,  
  "httpStatusCode": 400,  
  "message": "Invalid query parameter.",  
  "version": 2  
}
```

401 application/json (The user was not authenticated.)

```
{  
  "details": [  
    "correlator: 123e4567-e89b-12d3-a456-426614174000"  
  ],  
  "errorCode": 401,  
}
```

```
"httpStatusCode": 401,  
"message": "The credentials could not be validated.",  
"version": 2  
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{  
  "details": [  
    "correlator: 123e4567-e89b-12d3-a456-426614174000"  
  ],  
  "errorCode": 422,  
  "httpStatusCode": 422,  
  "message": "Missing or invalid fields in request body or parameters.",  
  "version": 2  
}
```

GET /sources/{sourceId}

Summary: Get a source

Operation ID: getSource

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getSource>

Returns a source object by its unique identifier.

Parameters

Name	Location	Type	Required	Description
sourceId	path	string	Yes	The id for the source object to get.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/  
const options = {  
  method: 'GET',  
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati  
};
```

```
try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
  }
  async with httpx.AsyncClient() as client:
    response = await client.get(url, headers=headers)
    print(response.json())
```

```
asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/sources/
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "id": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
  "inputType": "file",
  "insertTimestamp": "2025-12-12T19:48:53.448286+00:00",
  "metadata": {
    "createdBy": "9850832c-bf81-4335-9947-cdb9bb33d017"
  },
  "name": "Info Files",
  "secrets": {
    "envVars": []
  },
}
```

```
"sourceType": "local",
"updateTimestamp": "2025-12-13T09:15:22.123456+00:00"
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

GET /sources/{sourceId}/files

Summary: Get a list of files from a specific source

Operation ID: getSourceFiles

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getSourceFiles>

Returns a list of files from a local source.

Parameters

Name	Location	Type	Required	Description
sourceId	path	string	Yes	The id of the source to retrieve all files.
limit	query	['integer', 'null']	No	The maximum number of files to be returned.
start	query	integer	No	Zero based offset of the first file to be returned.
sortBy	query	['string', 'null']	No	The criteria for sorting. Supported attributes include sourceFileId, filepath, insertTimestamp, and updateTimestamp. Additional attributes might be added in future releases.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/  
const options = {  
  method: 'GET',  
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati  
};  
  
try {  
  const response = await fetch(url, options);  
  const data = await response.json();  
  console.log(data);
```

```
} catch (error) {  
    console.error(error);  
}
```

Python - Requests

```
import requests  
  
url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"  
headers = {  
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",  
    "Accept": "application/json",  
}  
  
try:  
    response = requests.get(url, headers=headers)  
    response.raise_for_status()  
    data = response.json()  
    print(data)  
except requests.exceptions.RequestException as error:  
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx  
import asyncio  
  
async def main():  
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"  
    headers = {  
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",  
        "Accept": "application/json",  
    }  
    async with httpx.AsyncClient() as client:  
        response = await client.get(url, headers=headers)  
        print(response.json())  
  
asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/sources/  
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \  
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{  
  "count": 3,  
  "items": [  
    {  
      "filepath": "example1.txt",  
      "insertTimestamp": "2025-12-12T10:15:30.123456+00:00",  
      "metadata": {  
        "size": 123,  
        "type": "text/plain"  
      },  
      "sourceFileId": "3c8b12a9-17d1-4f1b-822b-423ec8491bc0",  
      "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812",  
      "updateTimestamp": "2025-12-13T09:45:12.654321+00:00"  
    },  
    {  
      "filepath": "reports/2025-12-summary.pdf",
```

```

    "insertTimestamp": "2025-12-12T11:20:05.000000+00:00",
    "metadata": {
      "size": 204800,
      "type": "application/pdf"
    },
    "sourceFileId": "a1b2c3d4-1111-4f1b-822b-423ec8491bc0",
    "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
    "updateTimestamp": "2025-12-12T11:20:05.000000+00:00"
  },
  {
    "filepath": "data/weather/daily_2025-12-13.csv",
    "insertTimestamp": "2025-12-13T06:30:00.000000+00:00",
    "metadata": {
      "size": 9876,
      "type": "text/csv"
    },
    "sourceFileId": "b2c3d4e5-2222-4f1b-822b-423ec8491bc0",
    "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
    "updateTimestamp": "2025-12-13T06:45:12.000000+00:00"
  }
],
"limit": 10,
"name": "sourceFiles",
"start": 0
}

```

400 application/json (The request was invalid.)

```

{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}

```

401 application/json (The user was not authenticated.)

```

{
  "details": [

```

```
    "correlator": "123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator": "123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

POST /sources/{sourceId}/files

Summary: Create a file in a specific source

Operation ID: `createSourceFile`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/createSourceFile>

Uploads a file to a local source.

Parameters

Name	Location	Type	Required	Description
<code>sourceId</code>	path	<code>string</code>	Yes	The id of the source to upload a file to.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/';
const options = {
  method: 'POST',
```

```

headers: {
  Authorization: 'Bearer YOUR_ACCESS_TOKEN',
  'Content-Type': 'application/json',
  Accept: 'application/json'
},
body: JSON.stringify({
  "file": "example1.txt"
})
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source}"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
  "Content-Type": "application/json",
}
payload = {
  "file": "example1.txt"
}

try:
  response = requests.post(url, headers=headers, json=payload)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{s
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
        "Content-Type": "application/json",
    }
    payload = {
        "file": "example1.txt"
    }

    async with httpx.AsyncClient() as client:
        response = await client.post(url, headers=headers, json=payload)
        print(response.json())

asyncio.run(main())

```

Shell - cURL

```

curl -X POST "https://example.com/SASRetrievalAgentManager/api/v1/sources
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{
  "file": "example1.txt"
}'

```

Request Body – An example request for uploading files

```

{
  "file": "example1.txt"
}

```

Response Status Codes

Status Code	Description	Content-Types
201	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

201 application/json (The request succeeded.)

```
{
  "id": "3c8b12a9-17d1-4f1b-822b-423ec8491bc0",
  "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812"
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
```

```
    "correlator": "123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator": "123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

DELETE /sources/{sourceId}/files/{fileId}

Summary: Delete a file from a source

Operation ID: deleteSourceFile

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/deleteSourceFile>

Deletes a file in a local source.

Parameters

Name	Location	Type	Required	Description
sourceId	path	string	Yes	The id of the source to delete the file from.
fileId	path	string	Yes	The id of the file to delete.

Request Samples

JavaScript - Fetch

```

const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/
const options = {
  method: 'DELETE',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.delete(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",

```

```
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.delete(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X DELETE "https://example.com/SASRetrievalAgentManager/api/v1/source" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
204	The file was deleted.	None
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter."
```

```
"version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

GET /sources/{sourceId}/files/{fileId}

Summary: Get a file from a specific source

Operation ID: getSourceFile

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getSourceFile>

Returns a file in a local source.

Parameters

Name	Location	Type	Required	Description
sourceId	path	string	Yes	The id of the source to retrieve the file from.

Name	Location	Type	Required	Description
fileId	path	string	Yes	The id of the file to retrieve.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/'
const options = {
  method: 'GET',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{sourc
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{s
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.get(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/sources/
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "filepath": "report.pdf",
  "insertTimestamp": "2025-12-12T10:15:30.123456+00:00",
  "metadata": {
    "size": 204800,
    "type": "pdf"
  },
  "sourceFileId": "f39dad75-a389-469a-a258-4e7295a12b8b",
  "sourceId": "cef92c26-c0dc-4a6f-a10a-7e3996bf2559",
  "updateTimestamp": "2025-12-13T09:45:12.654321+00:00"
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
```

```
    "correlator": "123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

DELETE /sources/{sourceId}/files/{fileId}/tags/{tagId}

Summary: Removes a source file tag from a source file

Operation ID: deleteTagFromSourceFile

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/deleteTagFromSourceFile>

Removes a tag from a source file.

Parameters

Name	Location	Type	Required	Description
sourceId	path	string	Yes	The ID of the source the file belongs to.
fileId	path	string	Yes	The ID of the file to remove the tag from.
tagId	path	string	Yes	The ID of the tag to remove.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/'
const options = {
  method: 'DELETE',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati

}

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
```

```
    console.error(error);  
}
```

Python - Requests

```
import requests  
  
url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"  
headers = {  
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",  
    "Accept": "application/json",  
}  
  
try:  
    response = requests.delete(url, headers=headers)  
    response.raise_for_status()  
    data = response.json()  
    print(data)  
except requests.exceptions.RequestException as error:  
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx  
import asyncio  
  
async def main():  
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"  
    headers = {  
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",  
        "Accept": "application/json",  
    }  
    async with httpx.AsyncClient() as client:  
        response = await client.delete(url, headers=headers)  
        print(response.json())  
  
asyncio.run(main())
```

Shell - cURL

```
curl -X DELETE "https://example.com/SASRetrievalAgentManager/api/v1/sourc
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
204	The tag was removed.	None
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
```

```
"httpStatusCode": 401,  
"message": "The credentials could not be validated.",  
"version": 2  
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{  
  "details": [  
    "correlator: 123e4567-e89b-12d3-a456-426614174000"  
  ],  
  "errorCode": 422,  
  "httpStatusCode": 422,  
  "message": "Missing or invalid fields in request body or parameters.",  
  "version": 2  
}
```

PUT /sources/{sourceId}/files/{fileId}/tags/{tagId}

Summary: Update a source file with a tag

Operation ID: `updateFileWithTag`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/updateFileWithTag>

Updates a source file with a tag.

Parameters

Name	Location	Type	Required	Description
<code>sourceId</code>	path	<code>string</code>	Yes	The ID of the source the file belongs to.
<code>fileId</code>	path	<code>string</code>	Yes	The ID of the file to update the tag for.
<code>tagId</code>	path	<code>string</code>	Yes	The ID of the tag to update.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/  
const options = {
```

```

method: 'PUT',
headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.put(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
  }
  async with httpx.AsyncClient() as client:

```

```
response = await client.put(url, headers=headers)
print(response.json())
```

```
asyncio.run(main())
```

Shell - cURL

```
curl -X PUT "https://example.com/SASRetrievalAgentManager/api/v1/sources/"
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
404	The specified resource was not found.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "message": "Tag 'confidential' was successfully added to file with id "
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

404 application/json (The specified resource was not found.)

```
{
  "details": [
    "loc: path.id",
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 404,
  "httpStatusCode": 404,
  "message": "The specified resource was not found.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ]
}
```

```
],  
"errorCode": 422,  
"httpStatusCode": 422,  
"message": "Missing or invalid fields in request body or parameters.",  
"version": 2  
}
```

PUT /sources/{sourceId}/triggerAutomation

Summary: Trigger automation based on a source

Operation ID: `updateSourceTriggerAutomation`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/updateSourceTriggerAutomation>

Triggers automation for a source.

Parameters

Name	Location	Type	Required	Description
<code>sourceId</code>	path	<code>string</code>	Yes	The ID of the source to trigger its automation.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/sources/  
const options = {  
  method: 'PUT',  
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati  
};  
  
try {  
  const response = await fetch(url, options);  
  const data = await response.json();  
  console.log(data);  
} catch (error) {  
  console.error(error);  
}
```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
}

try:
    response = requests.put(url, headers=headers)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.put(url, headers=headers)
        print(response.json())

asyncio.run(main())

```

Shell - cURL

```

curl -X PUT "https://example.com/SASRetrievalAgentManager/api/v1/sources/{source_id}" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"

```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	None
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

Tags

GET /tags

Summary: Get a list of all source file tags

Operation ID: `getTags`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/getTags>

Returns a list of tags for source files.

Parameters

Name	Location	Type	Required	Description
<code>filter</code>	query	<code>['string', 'null']</code>	No	The criteria for filtering. Supported attributes include id, name, and sourceId. Additional attributes might be added in future releases.
<code>limit</code>	query	<code>integer</code>	No	The maximum number of records to be returned. <code>>= 1</code> or <code><= 100</code>
<code>start</code>	query	<code>integer</code>	No	The zero-based offset of the first record to be returned. <code>>= 0</code>

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/tags';
const options = {
```

```

method: 'GET',
headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'applicati
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}

```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/tags"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.get(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
except requests.exceptions.RequestException as error:
  print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
  url = "https://example.com/SASRetrievalAgentManager/api/v1/tags"
  headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
  }
  async with httpx.AsyncClient() as client:

```

```
response = await client.get(url, headers=headers)
print(response.json())
```

```
asyncio.run(main())
```

Shell - cURL

```
curl -X GET "https://example.com/SASRetrievalAgentManager/api/v1/tags" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
200	The request succeeded.	application/json
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

200 application/json (The request succeeded.)

```
{
  "count": 2,
  "items": [
    {
      "description": "Files that contain confidential information.",
      "id": "123e4567-e89b-12d3-a456-426614174000",
      "name": "confidential"
    },
    {
```

```
    "description": "Files related to project alpha.",
    "id": "123e4567-e89b-12d3-a456-426614174000",
    "name": "project-alpha",
    "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812"
  }
],
"limit": 10,
"name": "sourceFileTags",
"start": 0
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],

```

```
"errorCode": 422,  
"httpStatusCode": 422,  
"message": "Missing or invalid fields in request body or parameters.",  
"version": 2  
}
```

POST /tags

Summary: Create a source file tag

Operation ID: `createTag`

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/createTag>

Creates a tag for a source file.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/tags';  
const options = {  
  method: 'POST',  
  headers: {  
    Authorization: 'Bearer YOUR_ACCESS_TOKEN',  
    'Content-Type': 'application/json',  
    Accept: 'application/json'  
  },  
  body: JSON.stringify({  
    "description": "Files that contain confidential information.",  
    "metadata": {},  
    "name": "confidential"  
  })  
};  
  
try {  
  const response = await fetch(url, options);  
  const data = await response.json();  
  console.log(data);  
} catch (error) {  
  console.error(error);  
}
```

Python - Requests

```

import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/tags"
headers = {
    "Authorization": "Bearer YOUR_ACCESS_TOKEN",
    "Accept": "application/json",
    "Content-Type": "application/json",
}
payload = {
    "description": "Files that contain confidential information.",
    "metadata": {},
    "name": "confidential"
}

try:
    response = requests.post(url, headers=headers, json=payload)
    response.raise_for_status()
    data = response.json()
    print(data)
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')

```

Python - HTTPX (Async)

```

import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/tags"
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
        "Content-Type": "application/json",
    }
    payload = {
        "description": "Files that contain confidential information.",
        "metadata": {},
        "name": "confidential"
    }

    async with httpx.AsyncClient() as client:
        response = await client.post(url, headers=headers, json=payload)
        print(response.json())

```

```
asyncio.run(main())
```

Shell - cURL

```
curl -X POST "https://example.com/SASRetrievalAgentManager/api/v1/tags" \
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{
  "description": "Files that contain confidential information.",
  "metadata": {},
  "name": "confidential"
}'
```

Request Body – An example request for creating a global source file tag

```
{
  "description": "Files that contain confidential information.",
  "metadata": {},
  "name": "confidential"
}
```

Request Body – An example request for creating a local source file tag

```
{
  "description": "Files related to project alpha.",
  "metadata": {},
  "name": "project-alpha",
  "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812"
}
```

Response Status Codes

Status Code	Description	Content-Types
201	A tag was created.	application/json

Status Code	Description	Content-Types
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

201 application/json (A tag was created.)

```
{
  "description": "Files that contain confidential information.",
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "insertTimestamp": "2025-12-12T19:48:53.448286+00:00",
  "metadata": {},
  "name": "confidential",
  "updateTimestamp": "2025-12-12T19:48:53.448286+00:00"
}
```

201 application/json (A tag was created.)

```
{
  "description": "Files related to project alpha.",
  "id": "123e4567-e89b-12d3-a456-426614174000",
  "insertTimestamp": "2025-12-12T19:48:53.448286+00:00",
  "metadata": {},
  "name": "project-alpha",
  "sourceId": "079b46b9-fbc4-44ce-8cb1-163330e4d812",
  "updateTimestamp": "2025-12-12T19:48:53.448286+00:00"
}
```

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```

DELETE /tags/{tagId}

Summary: Delete a source file tag

Operation ID: deleteTag

Portal Link: <https://developer.sas.com/rest-apis/SASRetrievalAgentManager/deleteTag>

Deletes a source file tag.

Parameters

Name	Location	Type	Required	Description
tagId	path	string	Yes	The ID of the file tag to delete.

Request Samples

JavaScript - Fetch

```
const url = 'https://example.com/SASRetrievalAgentManager/api/v1/tags/{tagId}'
const options = {
  method: 'DELETE',
  headers: {Authorization: 'Bearer YOUR_ACCESS_TOKEN', Accept: 'application/json'}
};

try {
  const response = await fetch(url, options);
  const data = await response.json();
  console.log(data);
} catch (error) {
  console.error(error);
}
```

Python - Requests

```
import requests

url = "https://example.com/SASRetrievalAgentManager/api/v1/tags/{tagId}"
headers = {
  "Authorization": "Bearer YOUR_ACCESS_TOKEN",
  "Accept": "application/json",
}

try:
  response = requests.delete(url, headers=headers)
  response.raise_for_status()
  data = response.json()
  print(data)
```

```
except requests.exceptions.RequestException as error:
    print(f'Request failed: {error}')
```

Python - HTTPX (Async)

```
import httpx
import asyncio

async def main():
    url = "https://example.com/SASRetrievalAgentManager/api/v1/tags/{tagI
    headers = {
        "Authorization": "Bearer YOUR_ACCESS_TOKEN",
        "Accept": "application/json",
    }
    async with httpx.AsyncClient() as client:
        response = await client.delete(url, headers=headers)
        print(response.json())

asyncio.run(main())
```

Shell - cURL

```
curl -X DELETE "https://example.com/SASRetrievalAgentManager/api/v1/tags/
-H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
-H "Accept: application/json"
```

Response Status Codes

Status Code	Description	Content-Types
204	The tag was deleted.	None
400	The request was invalid.	application/json
401	The user was not authenticated.	application/json
422	The server understands the request but was unable to process the contained instructions.	application/json

Response Samples

400 application/json (The request was invalid.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 400,
  "httpStatusCode": 400,
  "message": "Invalid query parameter.",
  "version": 2
}
```

401 application/json (The user was not authenticated.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 401,
  "httpStatusCode": 401,
  "message": "The credentials could not be validated.",
  "version": 2
}
```

422 application/json (The server understands the request but was unable to process the contained instructions.)

```
{
  "details": [
    "correlator: 123e4567-e89b-12d3-a456-426614174000"
  ],
  "errorCode": 422,
  "httpStatusCode": 422,
  "message": "Missing or invalid fields in request body or parameters.",
  "version": 2
}
```
